An Interactive Extension of TriMap for Scalable and Insightful High-Dimensional Visualization

Benjamín Hučko, Camille Niessink, Viktória Pravdová, and Wojciech Trejter

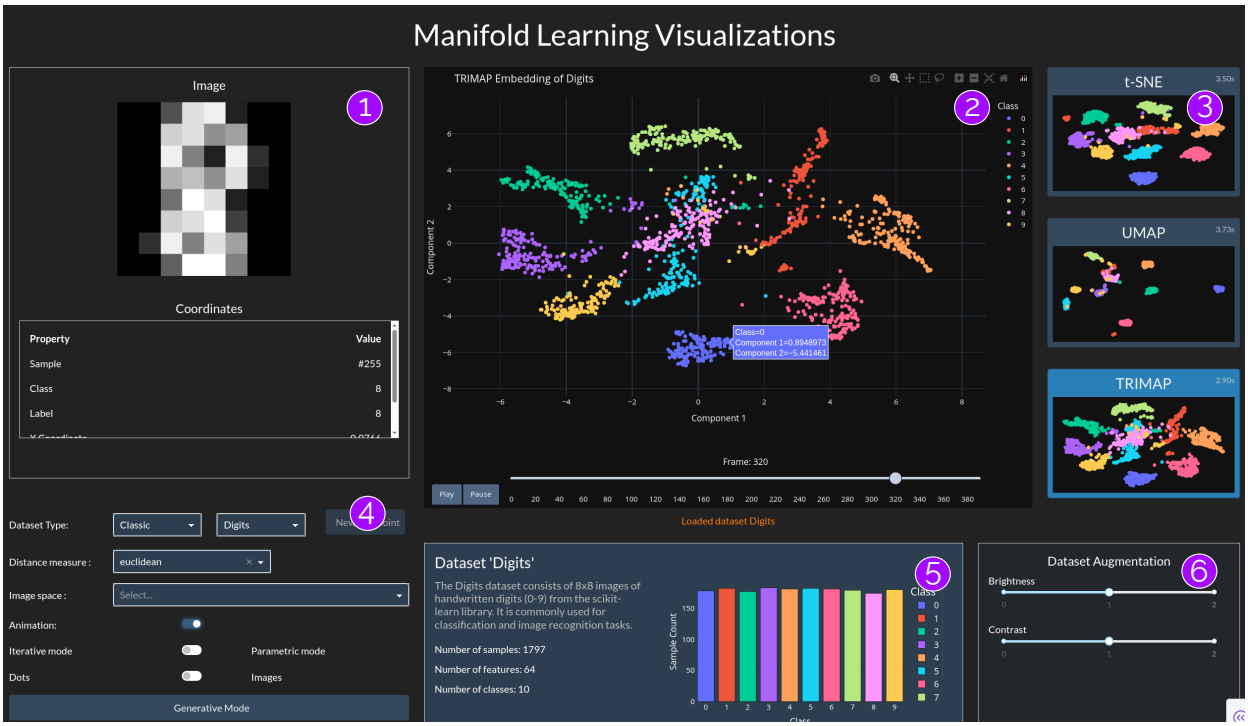

Fig. 1: Interactive visualization interface of our extended prototype TriMap system. Individual Datapoint Display (1) shows insight about an individual point chosen in (2). Main Interactive Graph (2) allows for interaction with the graph, with scaling, panning, category selection, and point interpolation options. Method Choice Thumbnails (3) allow for a selection of dimensionality reduction (DR) method, out of t-SNE, UMAP, and TriMap. Settings (4) allow for modification of data displayed in (2) and (3). Dataset Information (5) shows insight about the currently displayed dataset. Dataset Augmentation (6) enables modifications to the displayed dataset.

Abstract—
TriMap is a powerful dimensionality reduction technique that preserves global structure better than traditional methods like t-SNE and UMAP. However; it lacks advanced features such as dynamic updates, inverse transformation, and non-Euclidean distance support. This limits its applicability in interactive analytics settings. In this work, we extend TriMap with several capabilities, namely, enabling dynamic projection of new data points, inverse mapping from projected space back to the original space, a parametric neural variant for faster approximation, and support for a wide range of distance metrics beyond Euclidean. These extensions are integrated into a web-based multimedia analytics tool that supports real-time exploration and manipulation of high-dimensional datasets such as CIFAR-10, Fashion-MNIST, and PACS. We evaluate our system through both quantitative projection quality metrics and simulated user tasks, demonstrating that our enhancements significantly improve TriMap's usability and analytical depth for interactive machine learning workflows. .

1 Introduction

Interpreting and exploring high-dimensional data remains a central challenge in machine learning, particularly as models become increasingly complex. Dimensionality reduction techniques such as t-SNE and UMAP have long been used to visualize these data spaces [15, 20]. TriMap is a newer method offering efficient and scalable projections with strong preservation of global structure [1]. However, unlike UMAP, TriMap lacks extended functionality such as inversion, dynamic updates, and alternative geometries. These features are increasingly important for interactive data exploration. For instance, inverse transformation allows a user to compute the inverse mapping from the projection space back to the original high-dimensional data space, which help users to explore projections [4]. Meanwhile, dynamic updates enable integration of new data without retraining, making real-time or streaming analysis feasible [15].

In this work, our primary goal is to extend TriMap with novel functionality that makes it more suitable for interactive and practical use. We target machine learning researchers who need to explore relationships within complex datasets, investigate outliers, and generate new insights. We introduce support for hyperbolic geometry and custom distance functions to broaden its applicability. We also implement inversion and dynamic projections, enabling users to reconstruct data from the embedding space and to add new points without recomputing the full layout. These enhancements maintain TriMap's original scalability while significantly improving its flexibility.

To demonstrate the benefits of these extensions in a real-world setting, we integrate them into an interactive multimedia analytics tool. This system allows machine learning researchers to intuitively explore

high-dimensional datasets, analyze outliers, and compare projection methods. It supports popular datasets such as Digits [**?**], CIFAR-10 [11], Fashion-MNIST [22], and PACS [13], and provides a user-friendly interface for interactive analysis.

Our main contributions are:

- Extension of TriMap to support hyperbolic geometry and custom distance functions
- Addition of an inverse mapping from the projection space to the high-dimensional space
- Implementation of dynamic projections, allowing new data points to be added without retraining
- Integration of these extensions into an interactive tool for exploring embeddings across multiple datasets

2 Related Work

Dimensionality reduction is the process of transforming high-dimensional data into a lower-dimensional representation while retaining as much meaningful information as possible [21]. This technique is crucial for handling complex data efficiently, as it helps mitigate issues such as the curse of dimensionality and facilitates tasks such as classification, visualization, and compression. Traditional methods like Principal Component Analysis (PCA) [5] use linear transformations. These traditional methods are limited, because the intricate structures in real-world data cannot be fully captured with linear transformations. To better capture these structures, several non-linear techniques have been developed.

t-SNE (t-Distributed Stochastic Neighbor Embedding) is a popular method for reducing dimensionality and visualizing high-dimensional. It converts data point similarities in high-dimensional space into probabilities, and minimizing their divergence in the projected space. While effective for local structures, it faces high computational complexity and scalability issues [10, 20].

UMAP (Uniform Manifold Approximation and Projection) uses manifold theory to create a topological data representation, offering scalable and efficient dimensionality reduction. It matches t-SNE in visualization quality, often preserves more global structure, and with better runtime performance. This makes UMAP more versatile for various machine learning tasks [15].

In contrast, TriMap (Triplet-based Manifold Approximation and Projection) uses triplet constraints to capture higher-order relationships in the data. This helps maintain the overall structure, such as the placement of clusters and the identification of outliers. This means that TriMap produces embeddings that better reflect the global structure of the data compared to methods like t-SNE and UMAP. The triplet constraints are

$$\forall x,y,z \in \mathbb{R}^D, \|x-y\| < \|x-z\| \Rightarrow \|P(x)-P(y)\| < \|P(x)-P(z)\|,$$

where P is the TriMap projection to lower-dimensional space. TriMap provides a significantly better global view of the data than other dimensionality reduction methods such as t-SNE, LargeVis, and UMAP. TriMap shows excellent empirical performance on a variety of datasets by producing faithful projections with fast runtime. On performance benchmarks, TriMap easily scales to millions of points without depleting the memory and clearly outperforms t-SNE, LargeVis, and UMAP in terms of runtime [1].

Hyperbolic space embedding is valuable for dimensionality reduction methods due to its ability to efficiently represent hierarchical and complex data structures in low-dimensional spaces. Traditional Euclidean embeddings often struggle with the inherent hierarchical nature of many real-world datasets, leading to high distortion and loss of structural information when reducing dimensions. Hyperbolic space, with its negative curvature, can embed hierarchical data with significantly lower distortion, preserving both local and global relationships within the data. This makes it particularly useful for datasets that exhibit tree-like or network structures, such as social networks, biological taxonomies, and natural language processing tasks. By leveraging hyperbolic embeddings, dimensionality reduction methods can achieve more accurate and meaningful representations of complex data, enhancing the performance of downstream machine learning tasks [6, 17, 18].

3 Methodology

This section outlines the key extensions we introduced to enhance TriMap's capabilities for interactive high-dimensional data analysis. We present four main components: dynamic projections, inverse transformation, a parametric variant of TriMap, and support for non-Euclidean distances and spaces. These improvements address limitations in the original TriMap implementation and aim to make the method more suitable for multimedia analytics scenarios.

3.1 Dynamic Projections

Our approach to dynamic projections extends TriMap's functionality by enabling the addition of new points to existing embeddings without requiring complete recomputation. This methodology preserves the global structure of the original embedding while efficiently integrating new data points.

To embed a new point, we reuse TriMap's core idea, namely the optimization of triplet constraints. We keep the 2D positions of the original points fixed and only adjust the 2D position of the new point. The goal is to place the new point in such a way that it remains closer to its nearest neighbors (inliers) than to randomly chosen points (outliers), following the same triplet-based constraints used by TriMap. Triplets are constructed by selecting:

- a set of inliers: a point that is most similar (i.e., nearest neighbors) to the new point in the high-dimensional space
- a set of outliers: randomly selected points that are not among the inliers

Each triplet expresses a local similarity constraint of the form 'the new point is closer to the inlier than to the outlier'. This captures a basic intuition: if two data points are similar in the original space, they should also appear closer together in the 2D embedding.

To formalize this, we generate many such triplets and assign each one a weight based on how different the inlier and outlier are from the new point in the original space. These weighted constraints are used in a loss function, and we use gradient descent to find the best 2D position for the new point.

3.2 Inverse Transformation

Inverse transformation maps the data point from a low-dimensional TriMap projection back to the high-dimensional space. This allows the user to sample the low-dimensional embedding, and allows for greater insight about the dataset. Following the implementation of TriMap, the inverse transformation initializes the new points in the high-dimensional space. For the inverse transformations,

1. the projected points should map back to the original values, and
2. newly generated points should adhere to same TriMap constraints in the original space as in the projection.

TriMap inverse transformation can never satisfy the first constraint, because the initialization is inherently stochastic. In order to satisfy it, the inverse mapping initializes the previously mapped embeddings to their original values, and freezes their location. New embeddings are initialized by interpolating the original embeddings. To satisfy the second constraint, the inverse transformation minimizes TriMap loss for points in the original space.

3.3 Parametric TriMap

The implementations of inverse transformation and dynamic projection suffer from relatively large computation time and limited approximations. To improve on the two implementations, we train a neural approximation,

$$f_\theta : \mathbb{R}^D \to \mathbb{R}^d \quad (D > d),$$

subject to the TriMap constraint:

$$\forall x,y,z \in \mathbb{R}^D, \|x-y\| < \|x-z\| \Rightarrow \|f_\theta(x)-f_\theta(y)\| < \|f_\theta(x)-f_\theta(z)\|.$$

Parametric TriMap achieves the inverse transformation by training a decoder. Then it optimizes the combined loss;

$$L = (1-\alpha)L_{TriMap} + \alpha L_{recon},$$

where L_{recon} the Huber loss between the reconstructed projection and the original data, and α balances the losses to prevent one from dominating. The balancing parameter α plays an important role in combining the losses due to their different scale.

3.4 Non-Euclidean Distances and Spaces

Non-Euclidean distances play a crucial role in dimensionality reduction, particularly when dealing with complex, high-dimensional data that does not conform to the linear relationships assumed by Euclidean geometry. These distances, such as Manhattan, Cosine, or Haversine distances, can better capture the intrinsic structure and relationships within the data, leading to more accurate and meaningful reductions.

Natively, TriMap can generate triplets and assign their weights using one of 5 distance functions, those being Euclidean, Manhattan, Cosine, Hamming, and Chebyshev. However, the loss of the embeddings is only calculated based on Euclidean distance without any support for other distance functions.

Using UMAP as the current industry standard, we have converted its distance metrics to JAX, enabling the use of dozens of distance metrics used by PYNNDescent. We have also addressed the second problem of purely Euclidean-based loss, by enabling specification of an output metric to one of 18 defined metrics with gradients re-implemented from UMAP. Furthermore, using custom metrics is now possible by passing them as a callable variables in the invocation. We have enabled UMAP-like distance metrics which manually calculate both the loss and the gradient, although by default TriMap automatically calculates the gradient for the loss. With those options enabled, it is possible to use metrics such as Haversine distance, indicating the length of the shortest arc connecting two points on a sphere, or a hyperboloid distance, enabling casting the embeddings into hyperbolic spaces.

3.5 Reproduction of Results

The code is based off the JAX adaptation of TriMap[1]. To confirm that this adaptation of TriMap algorithm performs similarly to the original version and assess the diferences in inference time, we perform experiments which compare our adaptation of the JAX version with non-JAX TriMap, t-SNE, and UMAP.

We evaluate the inference time on standard multi-dimensional datasets such as COIL_20 [16], MNIST [3]. An attempt has been made to use RCV1 dataset [12], although UMAP has been the only method which did not exceed the available RAM memory. These datasets provide a diverse range of samples that help in assessing the performance and robustness of various machine learning algorithms. Additionally, they are widely recognized benchmarks in the field, making it easier to compare the results with other studies. Although this paper works on the JAX implementation of TriMap, the original Numba implementation is also evaluated with respect to its computation time. The experiments are run on an NVIDIA A100 GPU. In addition, 16 cores of Intel Xeon CPUs and 120GB of RAM were utilized.

We visualize how well the projections preserve both local and global structures with the Mammoth [2] and Swiss Roll dataset [14]. They are used to visually assess the capability of TriMap to preserve the global and local relationships in our adapted JAX implementation.

3.5.1 Projection evaluation

We compare the different projection techniques using cluster-label matching (CLM) [9], which evaluates the projection using label-trustworthiness (Label-T) and label-continuity (Label-C). Label-T&C compares the clustering structure in both the original space and the embedding. This allows for a more accurate assessment of how well the embedding preserves meaningful class relationships.

Label-T reflects class compression distortion, where lower scores indicate that points of different classes are closer in the embedding compared to the original data. On the other hand, Label-C captures class stretching distortion, where lower values suggest that points from different classes are more stretched in the embedding compared to the original data.

CLM calculates the clustering scores for both the high-dimensional space, and the projected space. We use two methods to calculate the clustering score: the between-dataset Calinski-Harabasz index (CH_{btwn}) [8] and Distance Consistency (DSC) [19].

We construct metric matrices

$$M_{ij} = \begin{cases} s(\{P_{L,i}, P_{L,j}\}, S, \delta) & \text{if } i \neq j \wedge s = CH_{btwn} \\ s(C(P_{L,i} \cup P_{L,j}, \delta), \{P_{L,i}, P_{L,j}\}) & \text{if } i \neq j \wedge s = DSC \\ 0 & i = j \end{cases},$$

where P_L corresponds to the ground truth label, C to the clustering algorithm, and δ To the distance function. We used the K-means clustering algorithm. The distance function for the evaluation is the same as distance metric used to create the projection. Thanks to our contribution, it is possible to compare TriMap to t-SNE and UMAP under different distance metrics. The LT and LC use

$$M^* = M(X) - M(P(X)),$$

where X is the original space and P is the projection. They aggregate the upper diagonal with

$$LT = 1 - avg_{i,j}(min(M^*_{i,j}, 0))$$
$$LC = 1 - avg_{i,j}(-max(M^*_{i,j}, 0)).$$

We calculate and report LT and LC for both CH_{btwn} and DSC.

4 Visual Analytics System

4.1 Layout

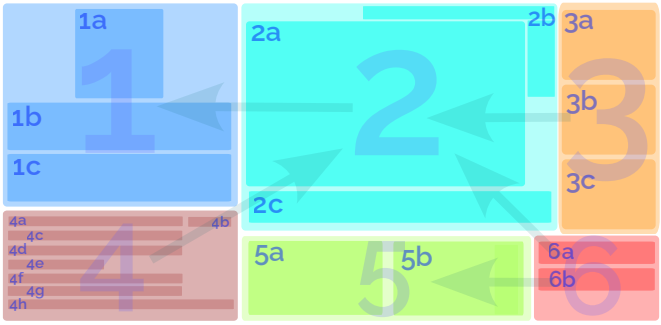

Fig. 2: Layout

Our design is centered around an interactive main plot (section 2a) of a chosen projection method (3a, 3b or 3c), see Fig. 2. This main visualization provides most of the initial high-level interactions, like differentiating data classes by color as well as the possibility to filter which of them to be displayed (2b), custom selection of areas of the plot that can be used for inter-plot comparisons (2b), zooming in on areas of the plot(2a), outlier detection (2a), and exploration of individual datapoints by clicking on them (2a).

The information about the clicked datapoint (section 1), including the image (1a), metadata (1c), and new embedding (1b), is immediately shown in the Datapoint Display in the top left corner of the screen. On the right, the user can see thumbnails of final embeddings produced by the 3 different methods (section 3), which not only provides a quick and easily accessible comparison between the methods but also work as buttons to choose which method to display in the main visualization.

The user can find information and statistics about the chosen dataset (5a) as well as a visualization of the dataset(5b), if applicable, at the bottom of the screen (section 5). The bottom right section also provides

[1] Base code available at https://github.com/google-research/google-research

low-level interactions like dataset augmentation sliders (section 6) (e.g., for brightness (6a) or contrast (6b)).

Most of the interactions happen in the bottom left corner (section 4), where most of the settings can be adjusted for lower-level interaction. The user can choose a dataset from the options (4a), choose which distance measure, including option for custom one, to use in the method (4c), whether to use a parametric or iterative approach (4e), which layer's activation to use as an input to trimap (4d) or whether to show some images amongst the dots in the main plot (4g). User can also try to add a new datapoint (4b) which activates a canvas to write in(1a), enter an animation mode (4e), which activates an interactive slider (2c) in the main plot or enter a generative mode (4h) which enables clicking also outside the known points in the main plot while displaying their expected images.

4.2 Interaction

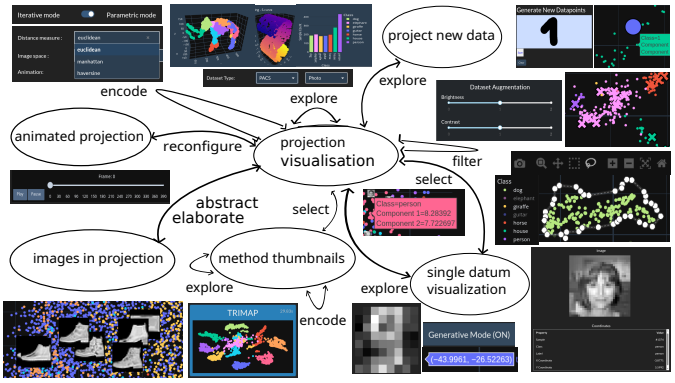

Fig. 3: Interaction diagram

The Main Graph and the individual datapoint display consist of multiple states. We simplify these states into an interaction diagram, which we use to evaluate how our visualization helps with high-level tasks.

Projection visualization is the initial state of the visualization. This state contains the 2D projection plots of all 3 projection techniques found in sections 2 and 3 of layout. In this state, the individual datapoint display in section 1 is empty. This state allows for following recursive actions:

- encode action, that projects with different distance function or performs parametric projection based on settings in section 4;
- filter action, that removes data with specified labels (2b); and
- explore action, that selects different datasets (4a).

Method thumbnails are a compact overview element located in section 3 of the layout. This state shows static previews of the three projection techniques (t-SNE, UMAP, TRIMAP), with each thumbnail rendered as a button that can be clicked to change the active embedding in the Main Graph (section 2). The thumbnails contain no axes, hover tooltips, or interactive controls, to reduce cognitive load and visual clutter.

This state enables the following actions:

- select action, by clicking on one of the thumbnails, which updates the Main Graph to display the corresponding projection;
- encode awareness, by displaying the time each method took to compute in the top-right corner of each thumbnail; and
- explore action, by visually comparing the layouts of different projections at a glance, helping the user quickly identify which method better separates or clusters their data.

Project new data (4b) adds additional datapoints to the Main Graph. The user adds the datapoints in two ways. Firstly, they add new datapoints by moving the data augmentation slider in section 6. Moving the slider samples 100 datapoints, augments them with the specified augmentation, and project them onto the Main Graph in section 2. Secondly, they draw the datapoints in a drawable canvas. By pressing an "Add new datapoint" button in the Settings (4b), the user is able to draw on a drawable canvas in section 1. The newly drawn datapoints are projected on the Main Graph. Both ways of adding new datapoints correspond to the explore action.

Single datum visualization is a state that mostly affects the Datapoint Display. In this state, the Datapoint Display renders the datapoint if it contains image data (1a), or lists its features if it contains non-image data (1c). Additionally, the Datapoint Display shows the coordinates and the label of the datapoint (2b). The user enters this mode with the

1. select action by clicking on a projected datapoint, and
2. explore action by entering the generative mode (4h) and clicking anywhere in the Main Graph.

Animated projection adds a slider (2c) in the Main interaction graph. This slider specifies the iteration of TriMap. The slider moves from an initial PCA projection, to the converged TriMap projection. Therefore, by moving the slider or by clicking the play button, the user gains insight on how the TriMap algorithm develops and projects the datapoints. The user enters this state with reconfigure action by clicking a button (4e) in the setting.

Images in projecton renders the projected datapoints in the Main Graph. The user enters this state with the elaborate action by clicking a button in the Settings.

4.3 Implementation

To develop a responsive and interactive visualization interface, we adopted a web-based architecture built on Dash by Plotly—a Python framework specifically designed for analytical web applications. This choice was motivated by Dash's seamless integration of Plotly for interactive visualizations and its support for server-side logic and a highly customizable, responsive user interface, all of which align well with the requirements of our project.

We utilized Plotly visualizations in three primary ways:

- **Main Graph** (Section 2): This central visualization offers rich interactivity, including class-based filtering, zooming, lasso selection, hover tooltips, and the ability to use custom images as datapoint icons—all natively supported by Plotly.
- **Method Thumbnails** (Section 3): These are simplified, non-interactive snapshots of the main graph, rendered without axes or toolbars and embedded within buttons to provide an intuitive overview.
- **Dataset Visualizations** (Section 5b): Depending on the dataset type, we used either a 3D interactive object viewer (for 3D data) or a bar chart showing class distributions (for other datasets).

The rest of the application interface was constructed using HTML elements for layout—defining section containers, buttons, dropdown menus and sliders, styled with CSS for visual design. Interactive behavior and logic were implemented using Dash callbacks, allowing dynamic updates and user-driven interaction throughout the application.

4.4 Evaluation

We evaluate the visualization with simulated users. The simulated users interact with the interaction state machine Fig. 3. The cost of each action depends on the complexity of the action and the length of calculation.

Using the costs for each actions that are in table 1 we evaluate an oracle simulated user on following high level tasks:

Task 1: Determine which label pairs have most similar data points,

Task 2: Detect outliers.

Task 3: Visualize class boundaries.

Task 4: Evaluate the effect of data augmentation on clustering.

Task 5: Determine most clear samples from each class.

Task 6: Deduplicate data.

State	Action	State	Cost
s_1	explore	s_1	5 (7)
s_1	encode	s_1	5
s_1	filter	s_1	0.2
s_1	explore	s_2	5 (1)
s_1	select	s_3	0.4
s_1	explore	s_3	5 (1)
s_1	reconfigure	s_4	5
s_1	elaborate	s_5	1

Table 1: Costs for evaluating the visualization with simulated agents. The states are projection visualization (s_1), project new data (s_2), single datum visualization (s_3), animated projection (s_4), and images in projection (s_5). *() cost for parametric mode*, returning to s_1 is simplified to cost 0

Task 7: Determine optimal number of iterations for TriMap.

These tasks require the simulated user to use most of the newly implemented features of TriMap, and evaluate whether our visualization efficiently integrates our backend.

5 Results

Tab. 2 shows the inference time of the analyzed methods. TriMap with automatic gradient is a few milliseconds slower than TriMap without our additions. Therefore, our version of TriMap with default settings does not significantly hinder the performance of the method. However, using manually defined gradient function slows down the process by 4 seconds for both MNIST and COIL_20. Although TriMap (Numba) is significantly faster than the JAX-based TriMaps, computing COIL_20 in just 1.5 seconds, being the fastest method tested and exceeding the other TriMaps by 30 seconds, For a larger dataset of MNIST, it is already slower than either of the JAX-based TriMaps. It results in a speedup of 0.62x for JAX and 0.48x for the original implementation, whereas [1] indicated a speedup of 0.90x.

The much lower speedup of TriMap over UMAP for MNIST dataset compared to the values reported in [1] most likely results from the overhead caused by using PynnDescent in TriMap for nearest neighbors search, as opposed to using Annoy in the Numba implementation. To conclude whether JAX TriMap is usually slower than the newest version of UMAP, further tests are necessary.

Dataset (size)	Method	Time (seconds)
COIL_20 (1440)	**TriMap (Numba)**	**1.5**
	TriMap manual gradient	36.6
	TriMap auto gradient	31.5
	UMAP	16.0
	t-SNE	3
MNIST (70k)	TriMap (Numba)	57.0
	TriMap manual gradient	51.0
	TriMap auto gradient	44.6
	UMAP	**27.6**
	t-SNE	194.7

Table 2: Time taken for different methods on MNIST and coil_20 datasets. Shortest time in bold.

In addition to runtime performance, we assess the quality of the dimensionality reduction methods using Label Trustworthiness and Continuity (Label-T&C). Table 3 presents Label-T and Label-C scores for TriMap, UMAP, and t-SNE across three datasets (Digits, MNIST, and Fashion-MNIST), and evaluated using two clustering validation measures (CVM), namely the between-dataset Calinski-Harabasz index (CH_{btwn}) and Distance Consistency (DSC). Overall, the results highlight that performance varies not only across methods but also depending on the chosen CVM.

When using CH_{btwn}, TriMap consistently achieves the highest Label-T scores for Digits and MNIST, indicating that it introduces the least compression distortion, meaning that the points from different classes remain well separated in the embedding, similar to their separation in the original space. However, it often shows lower Label-C scores compared to t-SNE and UMAP, suggesting that points within the same class become more distributed in the embedding, resulting in greater stretching distortion. In contrast, t-SNE frequently achieves the highest Label-C scores, particularly on Digits and MNIST.

Dataset	CVM	Method	Label-T	Label-C
Digits	CH_{btwn}	TriMap	**0.9843**	0.9077
		UMAP	0.9480	0.9217
		t-SNE	0.8537	**0.9335**
	DSC	TriMap	0.9450	**0.9943**
		UMAP	0.9643	0.9843
		t-SNE	**0.9792**	0.9889
MNIST	CH_{btwn}	TriMap	**0.9600**	0.8750
		UMAP	0.9330	0.9050
		t-SNE	0.8450	**0.9220**
	DSC	TriMap	0.7811	**1.0**
		UMAP	0.9999	0.9379
		t-SNE	**1.0**	0.9435
Fashion-MNIST	CH_{btwn}	TriMap	0.9068	**0.9993**
		UMAP	**0.9801**	0.9743
		t-SNE	0.9795	0.9904
	DSC	TriMap	0.8779	**0.9994**
		UMAP	**0.9692**	0.9533
		t-SNE	0.9645	0.9693

Table 3: Label-Trustworthiness (Label-T) and Label-Continuity (Label-C) scores for TriMap, UMAP, and t-SNE across multiple datasets using between-dataset Calinski-Harabasz index (CH_{btwn}) and Distance Consistency (DSC) as Clustering Validation Measures (CVM).

Under the DSC evaluation, the results differ: t-SNE and UMAP generally outperform TriMap on both Label-T and Label-C. For example, on MNIST, t-SNE achieves a perfect score, and UMAP reaches near-perfect scores for both metrics. This suggests that, when evaluated with DSC, t-SNE and UMAP produce embeddings that better preserve both the original separation between different classes and the compact structure of points belonging to the same class.

Furthermore, the evaluation with simulated users in table 4 shows that most high-level tasks are performed with few actions. This means our design of having each state reachable from the initial state in 1 action performs well on the tasks.

Task	Action description	Cost
1	filter pairs	$0.2 \times n$
2	Select sample	0.4
3	Generate samples near boundaries	$*1 \times n(n-1)$
4	Generate augmented samples	$*1$
5	filter and elaborate images	$(0.2+1) \times n$
6	encode into manhattan	5
7	reconfigure to animation	5

Table 4: Evaluation of our visualization with simulated users on high level tasks mentioned in pervious section. n is the number of classes. $* \rightarrow$ *cost when using parametric TriMap*

6 Case studies

We demonstrate four case studies to qualitatively analyze the capabilities of TriMap. We provide the main view of our visualization program for the Digits, Breast Cancer, Swiss roll, and Mammoth dataset. This selection consists of an image dataset, a non-image dataset, and two 3D datasets. Thus, it shows the generalization capabilities of the TriMap

dimensionality reduction technique. We do not do any preprocessing and apply the dimensionality reduction directly.

6.1 Digits

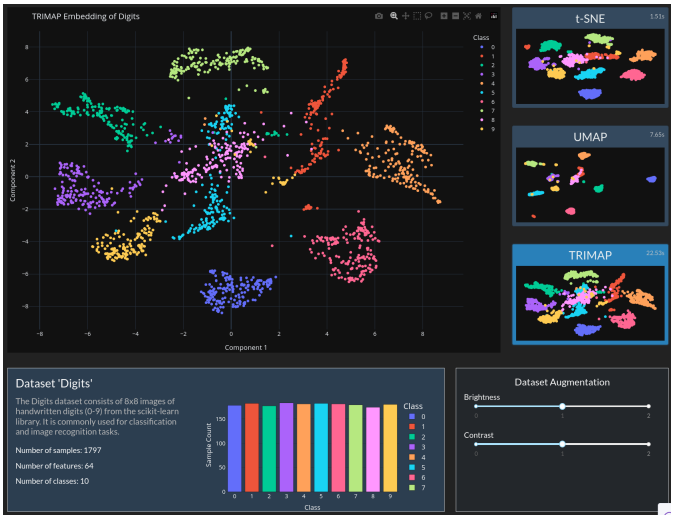

Fig. 4: Visual analytics system view of the digits dataset

The Digits dataset consists of hand written digits from 0 to 9. The dataset is balanced and stores each digit as an 8x8 grayscale image. We flatten the image before applying dimensionality reduction. Our visual analytics system shows that TriMap successfully clustered the dataset. The clusters are densely located in the space, unlike UMAP, which has more sparsely located clusters. Digits dataset embedded using TriMap,

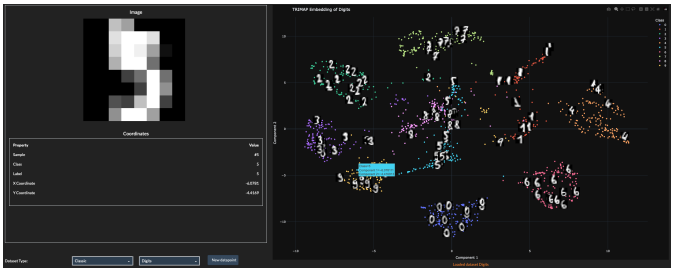

Fig. 5: Investigating outliers in Digits dataset

with 100 of the points accompanied by their graphical data is presented in Fig. 5. The Individual Datapoint Display (1) on the left part of the figure shows a point labeled as 5, but present in the cluster of 9s. Such outliers are distinguishable in the plot and allow for easier identification of outliers and their potential removal from the dataset, as their index in the dataset is also listed.
The option of displaying the images is turned on. Because of that, even if the target classes were not available, and thus the points would not be colored, the sets would be easily classifiable without the need to manually click the points on the plot.

As a simplistic showcase of the possibilities enabled by a wider access to distance metrics, Fig. 6 shows the same dataset of Digits embedded onto a Poincaré disk. The individual clusters are rather well separated. As there is no hierarchy to the categories, the clusters are all placed close to the border of the disk.

6.2 Breast Cancer

The breast cancer dataste consists of numeric features. These features correspond to 10 categories, and for each category there is a feature corresponding to the mean, standard error, and worst case. The dataset is skewed, and contains twice as many benign samples as malignant samples. TriMap projects this dataset in Fig. 7 as a single continuous

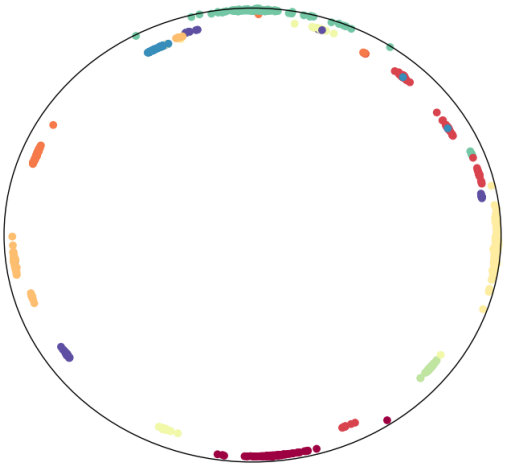

Fig. 6: Digits with Poincare disk model embedding

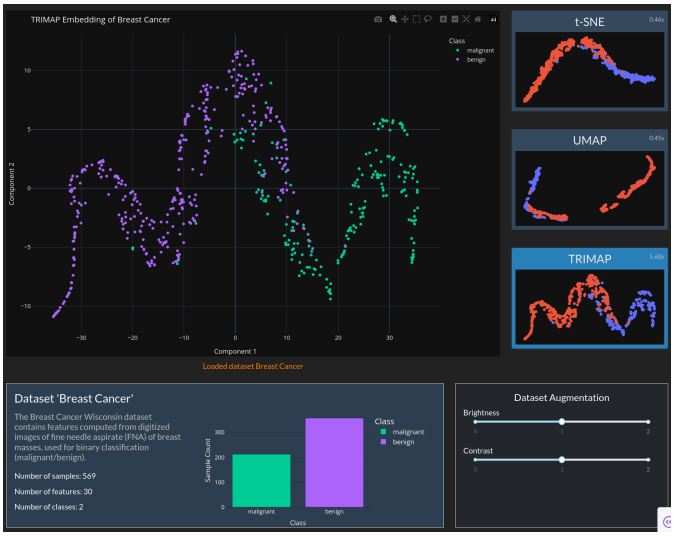

Fig. 7: Visual analytics system view of the breast cancer dataset

cluster. This is much more authentic than UMAP and t-SNE that cluster the dataset into two categories that do not correspond to the original labels. This shows that TriMap is able to deal with skewed datasets and does not form unnecessary clusters.

6.3 Mammoth

The Mammoth is a 3D visualization of a mammoth with optional number of points. The classes correspond to different body parts of the mammoth, e.g. tusk, foot, tail. TriMap projection in Fig. 8 keeps the correct global structure of the mammoth. The mammoth is clearly distinguishable and the classes are located in the correct locations. UMAP fully destroys the global structure, and the mammoth is not recognizable. T-SNE results in a more deformed version of the mammoth.

Fig. 9 shows the results of applying the dimensionality reduction techniques on the Mammoth 3D visualization with 100 000 points in a more comparative layout. In addition to the non-linear methods, we include PCA, as it is a starting point of TriMap. TriMap separates the tusks, but not to the extent visible in UMAP or t-SNE. Although the legs are distinguishable, they became crooked. Neither the structure of the head, nor the torso is clearly interpretable for any of the methods. It is clear that the result of TriMap is more deformed in the case of 100 000 points than in the case of 1 000 points. We believe this is due to the nature of the TriMap loss, which pushes away nearby projected points.

Fig. 8: Visual analytics system view of the Mammoth with 1000 points

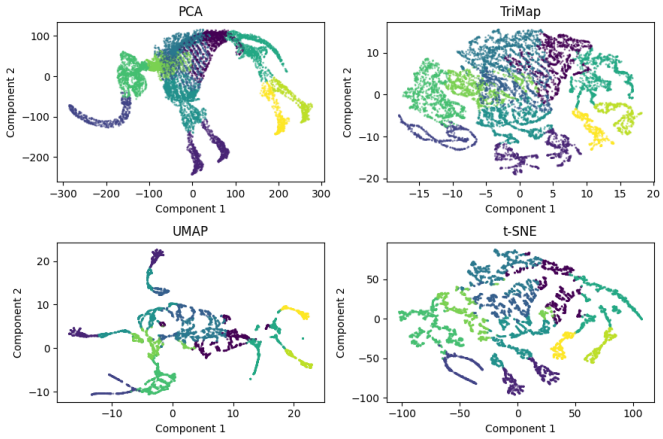

Fig. 9: Transformation of Mammoth dataset with 100 000 points into 2D space

6.4 Swiss Roll

The Swiss Roll is a 3D visualization of a spiral consisting of 1000 points. The Swiss Roll uses a continuous labels for the points. The labels correspond to the distance from the center of the spiral. TriMap projection in Fig. 10 successfully maintains the spiral structure, with just small deformation at the edge of the spiral. This deformation is caused by the points in the center of the spiral pushing away the points at the edge of the spiral. Both t-SNE and UMAP fail to represent the Swiss Roll as a spiral.

7 Discussion

7.1 Advantages of our extensions

In the results section we have shown the usefulness of dynamic projection and inverse transformation for a simulated user on several high level tasks. However, these features are much more impactful for the experience of a human user. The stochastic nature of dimensionality reduction makes it hard to work with visualization that completely changes when adding new data. Therefore, dynamic projections are not just a fancy method to save computation, but integral feature for any human user that wants to add extra data. Sampling the embedding space provides the user with additional freedom in the way how they want to explore. This is particularly important for human users, given that individuals acquire insight in diverse ways. Finally, showing the iterations in an animated way does not only tell the user what is optimal

Fig. 10: Visual analytics system view of the swiss roll

number of iteration for the algorithm, but also helps them to understand the dimensionality projection technique and become familiar with the visualization.

7.2 Limitations

Dynamic projection and inverse transformation are initialized using a liner interpolation in the target space of k nearest points in the source space. This is implemented as a K-Nearest Neighbors (KNN) search on the added and original points. This search is done in the projected space for inverse transformation and the original space for dynamic projections. This becomes problematic for batch inputs, in which all the KNN are the newly added inputs. This leads to zero initialization and poor performance. Therefore, the new points need to be either sufficiently spaced apart or the K in KNN needs to be larger than the size of the input batch. Both the dynamic projection and inverse transformation require caching the original data. This might creates significant memory overhead.
The parametric TriMap needs to balance between the TriMap loss and the reconstruction loss. These two losses are often not complementary, i.e. the model can either perform good TriMap or good reconstruction. Therefore, often one functionality of the parametric TriMap suffers.

7.3 Code inconsistencies

While developing the tools, we have found and addressed inconsistencies within the code. During the iterative part of TriMap, when the embeddings are optimized using gradient descent, a delta-bar-delta [7] approach is used. It returns the embeddings, velocity, and gain, but the values are assigned to embeddings, gain, and velocity, mixing the two values and destabilizing the convergence process. As the default hyperparameters of the algorithm were adapted to perform well with the error in the code, fixing it led us to modify the learning rate of the algorithm. We have found a learning rate of 1 leading to satisfactory results for haversine distance, but for the euclidean and manhattan distances, a learning rate of 100 is used. A further investigation on the need for such a large learning rate is advised if TriMap was to become the industry standard for dimensionality reduction.

8 Conclusion

TriMap offers state-of-the-art performance across variety of metrics and datasets. Our work extends TriMap with additional functionalities, which increase the number of situations where it can be used. Our visualization integrates these additional functionalities. This paper shows how these functionalities can be used for multiple high level task and proves with simulated users that our visualization provides an efficient interaction design. By adding the functionality and showing

its potential, this work achieves two-fold contribution to the field of dimensionality reduction.

References

[1] E. Amid and M. K. Warmuth. Trimap: Large-scale dimensionality reduction using triplets, 2022. 1, 2, 5

[2] A. Coenen and A. Pearce. Understanding umap, 2019. Accessed on 2025-06-12. 3

[3] L. Deng. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine*, 29(6):141–142, 2012. 3

[4] M. Espadoto, G. Appleby, A. Suh, D. Cashman, M. Li, C. Scheidegger, E. W. Anderson, R. Chang, and A. C. Telea. Unprojection: Leveraging inverse-projections for visual analytics of high-dimensional data. *IEEE Transactions on Visualization and Computer Graphics*, 29(2):1559–1572, 2021. 1

[5] K. P. F.R.S. Liii. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901. doi: 10.1080/14786440109462720 2

[6] Y. Guo, H. Guo, and S. Yu. Co-sne: Dimensionality reduction and visualization for hyperbolic data, 11 2021. doi: 10.48550/arXiv.2111.15037 2

[7] R. A. Jacobs. Increased rates of convergence through learning rate adaptation. *Neural networks*, 1(4):295–307, 1988. 7

[8] H. Jeon, M. Aupetit, D. Shin, A. Cho, S. Park, and J. Seo. Sanity check for external clustering validation benchmarks using internal validation measures. *arXiv preprint arXiv:2209.10042*, 2022. 3

[9] H. Jeon, Y.-H. Kuo, M. Aupetit, K.-L. Ma, and J. Seo. Classes are not clusters: Improving label-based evaluation of dimensionality reduction. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):781–791, 2023. 3

[10] D. Kobak and G. C. Linderman. Initialization is critical for preserving global data structure in both t-sne and umap. *Nature biotechnology*, 39(2):156–157, 2021. 2

[11] A. Krizhevsky, G. Hinton, et al. Learning multiple layers of features from tiny images. 2009. 2

[12] D. D. Lewis, Y. Yang, T. G. Rose, and F. Li. Rcv1: A new benchmark collection for text categorization research. *Journal of machine learning research*, 5(Apr):361–397, 2004. 3

[13] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales. Deeper, broader and artier domain generalization. In *Proceedings of the IEEE international conference on computer vision*, pp. 5542–5550, 2017. 2

[14] S. Marsland. *Machine learning: an algorithmic perspective*. Chapman and Hall/CRC, 2011. 3

[15] L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018. 1, 2

[16] S. A. Nene, S. K. Nayar, H. Murase, et al. Columbia object image library (coil-20). 1996. 3

[17] W. Peng, T. Varanka, A. Mostafa, H. Shi, and G. Zhao. Hyperbolic deep neural networks: A survey. *IEEE Transactions on pattern analysis and machine intelligence*, 44(12):10023–10044, 2021. 2

[18] F. Sala, C. De Sa, A. Gu, and C. Ré. Representation tradeoffs for hyperbolic embeddings. In *International conference on machine learning*, pp. 4460–4469. PMLR, 2018. 2

[19] M. Sips, B. Neubert, J. P. Lewis, and P. Hanrahan. Selecting good views of high-dimensional data using class consistency. In *Computer graphics forum*, vol. 28, pp. 831–838. Wiley Online Library, 2009. 3

[20] L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008. 1, 2

[21] L. Van Der Maaten, E. Postma, J. Van den Herik, et al. Dimensionality reduction: a comparative. *J Mach Learn Res*, 10(66-71), 2009. 2

[22] H. Xiao, K. Rasul, and R. Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017. 2